

Il faut choisir les sujets des phrases : mix de on, nous, vous
-> que des nous/vous ?

Comment faire fonctionner le robot

1/ Que y a-t-il dans le robot :

Le robot est équipé de :

- Scout Mini
- Bunker Mini
- Asus NUC 15 Pro CRK
- Router Teltonika RUT956
- GPS : simpleRTK2B Ardu Simple
- IMU : Phidgetspatial
- RealSense D435
- LiDAR: Robosense Helios 16P

manque un déterminant devant les pts OU reformule

2/ Comment allumer le robot :

Pour commencer, veuillez allumer le robot en appuyant sur le bouton de démarrage a l'arrière du robot. Une fois allumer le bouton s'allume en rouge et un écran avec la batterie et le voltage aussi :



3/ Télécommande

Pour télécommander le robot, prenez la manette. Vous disposez de deux joysticks et 4 interrupteurs dont les 2 intérieurs. Il vous faut maintenir les deux boutons power pendant quelques secondes, en ayant tous les interrupteurs en position haute. Après émissions des sons, la télécommande est allumée.



Le robot ne réagit pas au joystick. Pour le débloquer, veuillez abaisser le deuxième levier en partant de la droite de 1 cran. Ensuite, le troisième levier permet contrôler la vitesse :

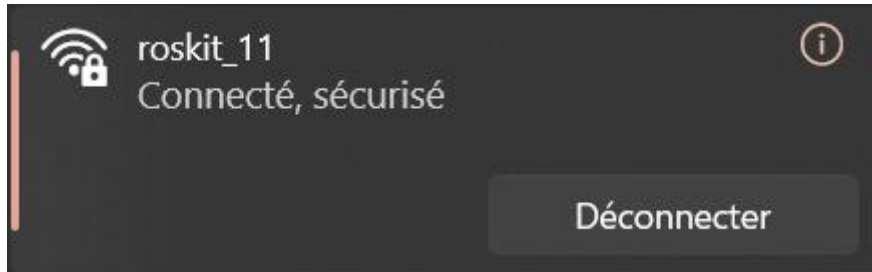
- Position du milieu -> vitesse lente
- Position du haute -> vitesse medium
- Position du basse -> vitesse haute

Utiliser les joysticks pour piloter le robot.

Vous pouvez utiliser le dernier levier pour contrôler le feu avant.

4/ Connexion wifi – SSH

Pour travailler sur le robot, attendre quelques secondes que le router s'allume. Chercher la connexion au robot en wifi. Le nom du robot est inscrit sur le robot, il devrait s'appeler roskit_ « numéro du robot ».



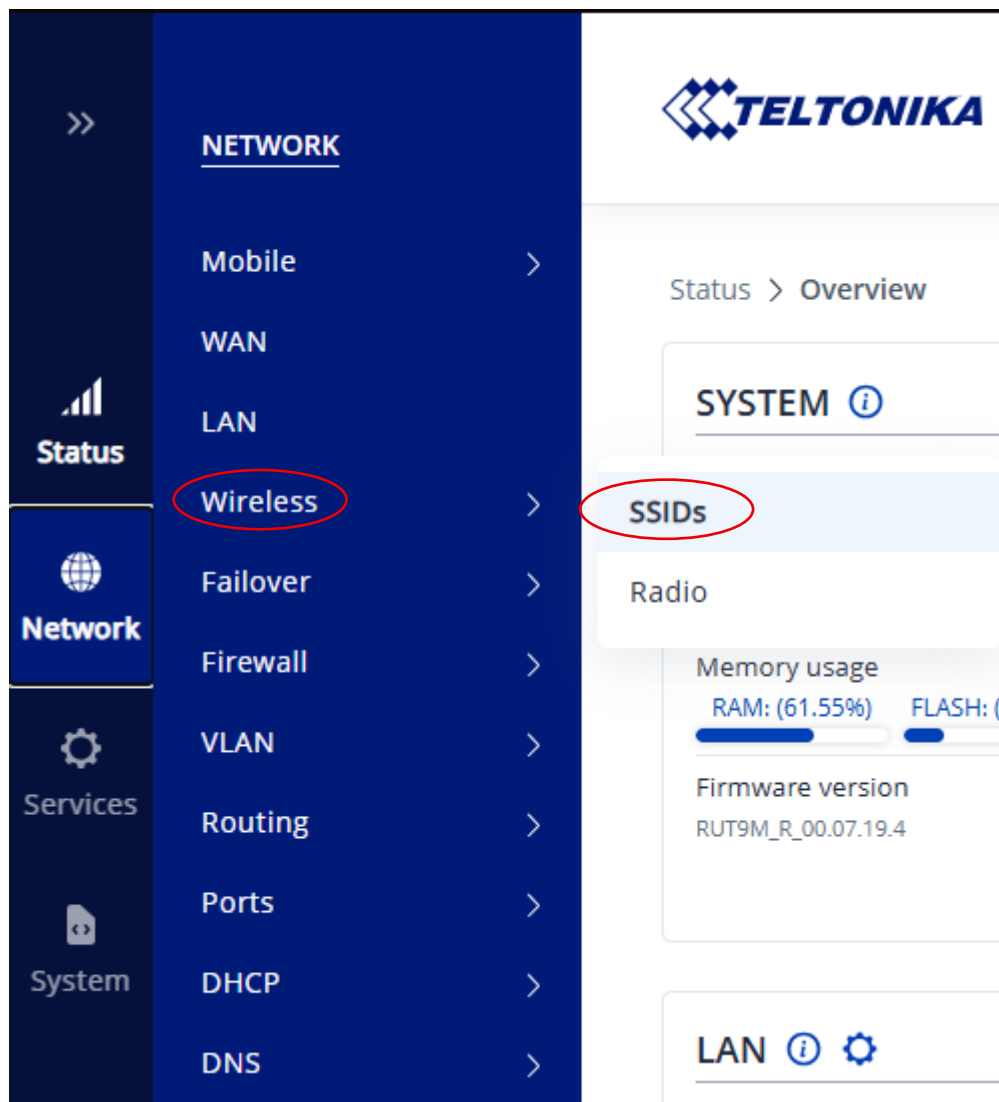
Si ce nom n'apparaît pas c'est que le robot a été réinitialisé, il faut aller sur le manuel d'utilisation et chercher le nom correspondant au robot. Une fois fait, je nous somme connecter au robot, mais nous n'avons pas internet.

Pour se connecter à internet, on va se connecter à <https://192.168.1.1> . On tombe sur cette page.

A screenshot of the Teltonika router's web interface. On the left, there is a blue sidebar with the Teltonika logo and the text 'Authorization required' and 'Please enter your username and password ?'. On the right, there is a white login form with two input fields: 'Username' and 'Password'. The 'Password' field has a toggle icon for visibility. Below the fields is a blue 'Log in' button.

Dans le manuel utilisateur, on y trouve les logins. Dans la section 6.1 : Router Page.

Une fois connecté au site, aller dans Network, Wireless puis SSIDs.



Puis cliquer sur Scan 2.4GHz, sélectionner la connexion que vous voulez utiliser. On ne peut pas se connecter à la connexion de l'école. Cela est due à la redirection nécessaire pour mettre ses codes, ce qui n'est pas faisable depuis le robot.

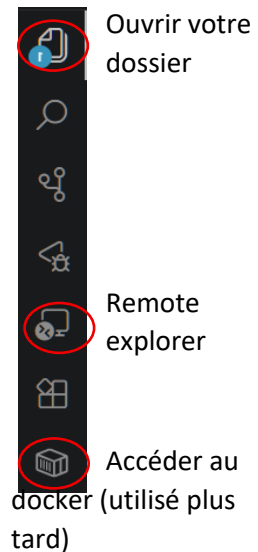
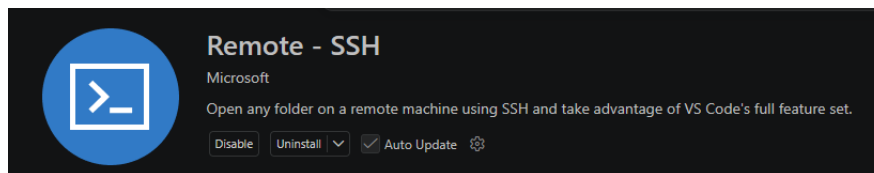
Ensuite, pour se connecter en SSH, deux options se présentent à vous.

Premièrement, connectez-vous à <https://192.168.1.102/> .

Le mot de passe pour tous les robots est : « ROSKit ». Vous êtes connecté sur l'ordinateur de bord du robot. Dans un terminal, vous pouvez taper « code . ». Une fenêtre VSCode devrait s'ouvrir. Cette méthode permet d'avoir VSCode dans noVNC.

[mettre un screen de noVNC ?](#)

Deuxièmement, depuis VSCode installer l'extension Remote -SSH :



Ensuite, rendez-vous dans l'onglet remote explorer en haut gauche :

Dans le remote explorer, appuyer sur le + pour ajouter une connexion en SSH.

Dans la barre de recherche taper : user@ « nom_du_robot ».local dans la forme suivant, exemples :

- user@grkit-007.local
- [user@grkit-011.local](#)

On vous demandera un mot de passe, c'est celui du robot. Encore une fois, pour tous les robots, le mot de passe est **ROSKitGR**. Si un message d'erreur s'affiche, réessayer, sinon continuer.

Pour continuer, sélectionner le répertoire dans lequel vous voulez travailler. Ici il s'agit de /home/user/roskit_ws/src. Une fois dedans vous verrez tous les package ros2 déjà installer par le fabricant.

Finalement, un docker est déjà construit et lancer au démarrage du robot. Vous pouvez donc vous y connecter via : « docker exec -it gr-p247-humble-container bash ».

Faire : **ros2 topic list**

S'il ne contient que 2 topic, alors faites la procédure ci-dessous, sinon poursuivre.

On source ros2 : **./opt/ros/humble/setup.bash**

On supprime les anciens dossiers : **rm -rf build/ install/ log/**

On relance le build proprement : **cm**

On install : **./install/setup.sh**

Et finalement on démarre tout : **ros2 launch gr_bringup main.launch.py**

Ce dernier launch permet d'émettre tous les topics, s'il faut, laissez le tourner en fond.

Vous voici prêt à travailler dans de bonnes conditions.

5/ Navigation Autonome

Toute cette partie est à réaliser dans des dockers.

Avant toute chose, sourcez votre docker avec **source /opt/ros/humble/setup.bash**, si ce n'est pas déjà fait.

Pour la navigation, nous allons utiliser nav2. Le but est de pouvoir demander au robot de se rendre à un point donné en temps réel.

Outils nécessaires :

```
sudo apt update  
sudo apt install -y \ ros-humble-navigation2 \ ros-humble-nav2-bringup \  
ros-humble-pointcloud-to-laserscan \ ros-humble-slam-toolbox
```

Le robot renvoie un nuage de point 3D que nav2 n'ait pas capable de traiter. Il faut donc faire la conversion des points 3D en points 2D pour que le robot sache à quelle distance les objets sont de lui. Pour cela, voir sur le github, dans lignes_de_commandes.

Une fois la commande exécuter, le robot devrait maintenant publier dans le topic /scan son nuage de point à 30cm du sol : **ros2 topic echo /scan**

Vérifier que les données soient bien transmises.

Ensuite, afin de naviguer, nav2 a besoin d'une carte. Pour l'instant nous n'en avons pas. Nous allons utiliser slam_toolbox pour simuler une carte à partir de l'odométrie du robot et du laserscan récupéré juste au-dessus. L'odométrie permet d'estimer la position du robot très rapidement mais reste imprécise, donc nous la corrigeons avec les données du laserscan.

Script permettant d'obtenir cette map (la commande est à réaliser dans un terminal en parallèle de la précédente):

```
ros2 launch slam_toolbox online_async_launch.py use_sim_time:=False
```

Il se peut que vous rencontriez des problèmes liés à l'odométrie. Veuillez vérifier que la base frame est bien base_link. En effet, l'odométrie est convertie en base_link, mais les paramètres peuvent chercher à un autre nom, résultant en un problème.

Dans le docker, faire coulisser et aller à

```
Files\opt\ros\humble\share\slam_toolbox\config\mapper_params_online_async.yaml
```

Chercher dans:

```
1  slam_toolbox:
2    ros_parameters:
3
4    # Plugin params
5    solver_plugin: solver_plugins::CeresSolver
6    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
7    ceres_preconditioner: SCHUR_JACOBI
8    ceres_trust_strategy: LEVENBERG_MARQUARDT
9    ceres_dogleg_type: TRADITIONAL_DOGLEG
10   ceres_loss_function: None
11
12   # ROS Parameters
13   odom_frame: odom
14   map_frame: map
15   base_frame: base_link
16   scan_topic: /scan
17   use_map_saver: true
18   mode: mapping #localization
```

Chercher la base_frame, et vérifier son nom.

Maintenant nous allons passer à la dernière étape. Il est possible que votre nav2_params.yaml ne soit pas valide. Rendez-vous dans le fichier suivant :

`\opt\ros\humble\share\nav2_bringup\params\nav2_params.yaml`

S'il diffère de celui donné ci-dessous, remplacer le. Celui que vous avez est celui téléchargé par défaut, il contient certaine faute de frappe et partie que nous n'utilisons pas. Voici une version qui correspond à notre usage.

https://github.com/MatthiasVermot-Desroches/Polytech-Scout-Agilex-GR-TP/blob/main/TP/TP1/nav2_params.yaml

Maintenant, nous devrions être prêt pour lancer la navigation :

`ros2 launch nav2_bringup navigation_launch.py`

Rendez-vous sur noVNC, dans rviz vous devriez avoir la possibilité de demander au robot d'aller une position dans les options en haut :



Dans une zone clairement défini par le Lidar de, cliquer et maintenant enfoncé à l'endroit et dans l'orientation dans lequel vous souhaitez que votre robot se rende. Une fois la flèche orientée, vous pouvez relâcher et si la destination est atteignable selon le robot, il s'y rendra. N'ayant pas de carte pour l'instant, on ne peut pas lui demander d'atteindre de trop grande distance.

6/ Création et utilisation de carte :

Pour visualiser dans une carte, on reprend les 3 dernières commande et on affiche dans un nouvel onglet rviz dédiés, fournis par nav2.

Rendez-vous dans rviz, **ADD -> By Topic -> chercher /map et cliquer sur Map**. Puis ok.

On veut enregistrer notre carte, pour faire cela, on garde tous les terminaux ouverts, et dans un nouveaux on tape :

```
ros2 run nav2_map_server map_saver_cli -f nom_de_la_carte
```

dans le nav2_params.yaml, vous pouvez chercher cost_scaling_factor inflation_radius. Pour passer les portes. Vous pouvez réduire l'inflation_radius à 0.55 par exemple, et augmenter le cost_scaling_factor à 10. Attention en jouant avec ces paramètres, vous augmentez les possibilités de mouvements.

Liens Nav2 :

https://github.com/ros-navigation/navigation2/blob/main/nav2_map_server/README.md

https://github.com/SteveMacenski/slam_toolbox/blob/ros2

Pour le nom des packages/librairies style RVIZ ou Nav2, ptet les mettre en maj ou en italique